

AmogaDS (@amoga/ui → @amogads/ui)

Central Design System Architecture, Developer Integration & Automated Cross-Repository Upgrade Operations Guide

Package Name: @amogads/ui	Current Release: v1.0.1 (Public NPM)	Environment: Next.js 14-16 / React 18-19	Automation: GitHub Actions / Token Auth
---------------------------	---	---	--

1. System Architecture & Boundaries

AmogaDS is the centralized design system package powering all Amoga enterprise applications. It decouples core UI primitives, tokens, and reusable templates into a versioned, published NPM package (@amogads/ui), while providing a live interactive Next.js gallery for state inspection and component documentation.

Layer	Exports / Scope	Key Responsibilities & Components	Strict Boundary Rules
Design Tokens	@amogads/ui/theme.css @amogads/ui/tokens	OKLCH semantic colors, radius scales, font variables, elevation shadows, keyframe animations.	Zero hardcoded hex values. Must support light & dark modes seamlessly.
Core UI Primitives	@amogads/ui	57+ headless & styled Radix primitives: Button, Input, Card, Dialog, Sheet, Drawer, Table, Tabs, Select, Badge, Avatar, Sonner, Toast, DropdownMenu, etc.	Pure presentation. ZERO API calls, store bindings, or domain business logic.
Business Patterns	@amogads/ui	PageHeader, DataTable, StatusBadge, FilterBar, MetricCard, ConfirmDialog, PasswordInput, DatePicker, QrCodeDisplay, Stats01.	Multi-app reusable. Controlled purely via props and event callbacks.
Page Templates	@amogads/ui	ListTemplate, DetailTemplate, FormTemplate, DashboardTemplate, WizardTemplate, WorkspaceTemplate.	Full-page layout blueprints with slots for headers, toolbars, and content.

2. Consuming @amogads/ui in Next.js Applications

Integrating @amogads/ui into any Next.js (App Router or Pages Router) project requires 3 standardized steps:

```
# Step 1: Install from public NPM registry
npm install @amogads/ui

# Step 2: In app/globals.css (or src/styles/globals.css):
@import "tailwindcss";
@import "@amogads/ui/theme.css";

/* ■ CRITICAL: Scan compiled package for Tailwind v4 utility classes */
@source "../node_modules/@amogads/ui/dist";
@custom-variant dark (&:is(.dark, .dark *));
```

Step 3: Component Consumption Example in Code

```
import { Button, Card, DataTable, PageHeader, StatusBadge, ListTemplate } from '@amogads/ui'

export default function UsersPage() {
  return (
    <div>
      <PageHeader>
        <div>
          <div>title="User Management"</div>
          <div>description="Manage active system administrators and role assignments."</div>
          <div>actions={Invite New User}</div>
        </div>
      <div>
        <div>>
          <div>Production Gateway Active</div>
        </div>
      </div>
    </div>
  )
}
```

3. Cross-Repository Automated Upgrade Pipeline

AmogaDS features an enterprise-grade, controlled cross-repository upgrade engine. When new versions of @amogads/ui are published, downstream applications receive automated Pull Requests without manual intervention or risk of code regression.

Phase	Action Executed	Safety Guard & Enforcement Mechanism
1. Trigger	Push to main when package.json version changes or manual GitHub Action dispatch.	Path-filtered CI ensures workflow only executes when release version changes.
2. Discovery	Reads consumers-registry.json to discover active consuming repositories.	Only processes consumers where automationStatus == 'enabled'.
3. Branching	Clones consumer repo and creates isolation branch: chore/upgrade-amoga-ui-vX.Y.Z.	Uses scoped GitHub Token / App auth. Zero hardcoded credentials.
4. Code Guard	Updates "@amogads/ui": "^X.Y.Z" in consumer's package.json.	STRICT FILE BOUNDARY: If any file outside package.json or lockfiles is touched, job immediately aborts.
5. PR Dispatch	Pushes branch and opens detailed GitHub PR on consumer repository.	PR includes changelog, safety checklist, testing instructions, and rollback command.
6. Human Review	Consuming app owner reviews CI status and merges manually.	ZERO AUTO-MERGE: Requires explicit repository maintainer approval.

4. Central Consumer Registry & Status Dashboard

Maintainers can inspect the real-time upgrade and compliance status across all registered consumer applications using: `npm run consumers:status` from the `amogads/` directory.

Repository	App Name	Installed	Target	Status	Automation
MohammadAmannn/amoga-pilot-app	Amoga Pilot App	v1.0.1	v1.0.1	■ up-to-date	enabled
MohammadAmannn/shadcn-admin	Shadcn Admin Pilot App	v1.0.1	v1.0.1	■ up-to-date	paused
MohammadAmannn/amoga-analytics	Amoga Analytics Portal	v1.0.1	v1.0.1	■ up-to-date	paused

5. Customization Safety & Non-Destructive Upgrades

Will package updates overwrite custom code in consuming projects?

NO. Consumer code is 100% protected. Here is how custom code behaves across all 3 customization models:

- **1. Custom className Passing:** Primitives use `cn(variants, className)` with `tailwind-merge`. Custom utility classes passed by the consumer always override default package styles without altering package source.
- **2. Composite Wrapper Components:** Custom components written in consumer folders (e.g. `src/components/custom/UserCard.tsx`) reside completely inside the consumer application. Upgrades never touch consumer source folders.
- **3. Additive Prop Contract:** All new props added to AmogaDS components must be optional (`prop?: type`) with sensible default values, ensuring existing component invocations never fail.
- **4. Emergency Rollback:** If any regression occurs after merging an upgrade, the consumer can roll back in 1 command: `npm install @amogads/ui@<PREVIOUS_VERSION>`.

6. Frequently Asked Questions (FAQ) & Operations Guide

Q1: Does pushing changes to the central amogads repository automatically modify or break my live consumer apps?

No. Consuming applications do NOT read from Git commits. They consume version-locked NPM packages installed in node_modules. Pushing code to amogads has **0% impact** on consuming applications until you build, version-bump, and publish a new NPM package.

Q2: Do I need to update consumers-registry.json manually every time I change code?

No. You only register a consuming application in consumers-registry.json **ONCE** when you first onboard that project. After that, the automated upgrade engine and tracker manage versioning, status updates, and PR links automatically.

Q3: How do automated upgrade PRs get triggered when I push from amogads?

The GitHub Actions workflow (.github/workflows/upgrade-consumers.yml) is configured to trigger automatically on push to the main branch whenever package.json is modified. When you bump your version and push, GitHub Actions automatically executes the upgrade engine and opens PRs across all active consumer repos.

Q4: How do I switch an existing project from @mohdaman/ui to @amogads/ui?

Execute three simple steps: (1) Run npm uninstall @mohdaman/ui && npm install @amogads/ui. (2) In globals.css, update the theme import to @import "@amogads/ui/theme.css"; and source to @source "../node_modules/@amogads/ui/dist";. (3) Do a global search and replace from '@mohdaman/ui' to '@amogads/ui' across your code.

Q5: Why was amogads/ added to .gitignore in the parent workspace?

Because amogads is an independent Git repository with its own remote (git@github.com:MohammadAmannn/amogads.git). Adding amogads/ to the parent .gitignore prevents Git from treating it as a nested submodule, ensuring both repositories stay completely clean and separated.

Q6: Where can developers find the complete component gallery and code snippets?

The repository includes a dedicated **HOW-TO-USE.md** developer guide with copy-pasteable snippets for all primitives, business components, message & chat cards, dialogs, and page templates. Developers can also run npm run dev in amogads to preview the live interactive gallery at <http://localhost:3000>.
